Phase 3: Project Design

1. System Architecture

The application follows a Client-Server Architecture: * **Client Side (Frontend):** Developed using standard web technologies (HTML, CSS). It provides the interface for data entry and result visualization. * **Server Side (Backend):** A Python Flask application (`app/app.py`). It handles incoming HTTP requests, preprocesses the input data to match the model's expectations, and passes the data to the ML model. * **Machine Learning Model:** A serialized XGBoost model (`app/models/*.pkl`) that is loaded into memory by the Flask app.

2. Directory Structure Strategy

To ensure continuous deployment works seamlessly (e.g., on Heroku), the core files (`Procfile`, `requirements.txt`, `run.py`) must remain in the root directory.

3. Database Schema (Conceptual ER Diagram)

Entity Name	Attributes	Description
Users	UserID (PK), Name, Email, Password, Role	For potential authentication.
ML_Model	ModelID (PK), ModelName, AlgorithmType, Accuracy, ModelFile	Metadata about the trained model.
Weather_Data	DataID (PK), UserID (FK), AnnualRainfall, CloudVisibility, Temperature, Humidity, SeasonalRainfall	The input parameters.
Prediction_Result	PredictionID (PK), DataID (FK), ModelID (FK), FloodResult, FloodProbability, PredictionDate	The output of the model.

4. UI/UX Design Flow

1. **Home Page:** Brief introduction and a button to access the prediction tool.
2. **Input Form:** A styled HTML form requiring specific numeric inputs for weather parameters.
3. **Result Page:** Displays the predicted outcome ("Flood" or "No Flood") prominently, along with a percentage indicating confidence/probability.